AMRUTVAHINI COLLEGE OF ENGINEERING, SANGAMNER

(An Autonomous Institute)

S.Y. B. Tech (Electronics and Computer Engineering)

CCE – CASE STUDY, Semester – III, Oct. 2026

Data Structures and Algorithms [R25-EC-PCC-202]

Case Study - Music Playlist Manager (08 Marks)

A music player app maintains a list of songs where users can add, delete, and play songs sequentially.

Implement Music Playlist Manager using doubly linked list. Reference implementation of basic doubly linked list is given.

Describe in short, your implementation with the help of diagrams (refer Diagram Requirements**)**

Test and validate your implementation using VS Code, and maintain the complete source code and related files in a GitHub repository. Commit your work regularly to document the development and progress of your implementation.

The task should be completed by a group of 4–5 students. However, each student will be evaluated individually based on their understanding, contribution, implementation, and performance during the demonstration and viva.

Note about Reference implementation –

Reference implementation is provided at the end to help you understand the expected coding style, variable naming convention, and pointer manipulation methodology. Use it as a guide to develop your own solution for the Music Playlist Manager. Follow the same programming methodology and pointer manipulation approach used in the given reference implementation.

Additional Instructions-

- You need to retain the same variable naming convention as per reference implementation:
  - `p` – use `p` to point to a newly allocated node or the node to be deleted.
  - `q` – use `q` as the navigation/traversal pointer to move through the linked list.
  - `head` – points to the first node (first song) in the playlist.
  - `current` – points to the currently selected/playing song.
- Modify the `Node` class appropriately to represent a song instead of a generic data value. For example, each node may store:
  - Song Title
  - Artist Name
  - `prev` pointer
  - `next` pointer
- Use the `prev` and `next` pointers of the doubly linked list to support navigation in both directions:
  - `current = current->next` → move to the Next Song
  - `current = current->prev` → move to the Previous Song

- The `current` pointer should be used to demonstrate the actual Next/Previous functionality of a music player. For example, if the current song is `Believer`:
    - Next → `Faded`
    - Previous → the song before `Believer`
- Implement a menu-driven Music Playlist Manager. The menu should provide appropriate options such as:

    1. Add Song
    2. Delete Song
    3. Display Playlist
    4. Display Playlist in Reverse
    5. Play Current Song
    6. Play Next Song
    7. Play Previous Song
    8. Play Complete Playlist
    9. Repeat All
    10. Show Current Song
    11. Exit
- Use appropriate pointer manipulation for:
    - Adding a new song
    - Deleting a song
    - Selecting the current song
    - Moving to the next song
    - Moving to the previous song
    - Displaying the playlist in forward direction
    - Displaying the playlist in reverse direction
- For Repeat All, restart playback from the first song (`head`) and play the complete playlist for the specified number of repetitions.
- Add necessary comments to your program to clearly explain the purpose and use of:
    - `p`
    - `q`
    - `head`
    - `current`
    - `prev`
    - `next`
- You may add new functions or modify existing functions wherever necessary to implement the required playlist features.
- Do not simply copy the reference program. Understand the pointer operations and adapt them to the Music Playlist Manager requirements.

Testing Requirements

Test your program with different situations, including:

- Empty playlist
- One song
- Multiple songs
- Adding songs at different stages
- Deleting the first song
- Deleting a middle song
- Deleting the last song
- Attempting to delete a song that does not exist
- Playing the current song

- Moving to the next song
- Moving to the previous song
- Attempting to move next from the last song
- Attempting to move previous from the first song
- Displaying the playlist in forward direction
- Displaying the playlist in reverse direction
- Repeating the complete playlist
- Exiting the menu properly

Important

The main objective of this case study is to demonstrate your understanding of the Doubly Linked List, particularly:

Node creation → Insertion → Deletion → Traversal → Bidirectional Navigation → Current Node Management

Your implementation should clearly demonstrate how the `prev` and `next` pointers make Next and Previous navigation possible.

Diagram Requirements-

Students must prepare and submit linked-list diagrams to demonstrate their understanding of the data structure and the pointer operations used in their program.

The submitted diagrams should include:

1. Initial Playlist – Show the `head`, `current`, `prev`, and `next` pointers for a playlist containing multiple songs.
2. Insertion – Show the linked-list structure before and after adding a new song, clearly indicating the changes in `prev` and `next` links.
3. Deletion – Show the linked-list structure before and after deleting a song, including deletion of a node and reconnection of the remaining nodes.
4. Next Navigation – Show how `current` moves from one song to the next using:
   `current = current->next`
5. Previous Navigation – Show how `current` moves from one song to the previous song using:
   `current = current->prev`
6. Reverse Traversal – Show how the playlist can be traversed in the reverse direction using the `prev` pointer.
7. Special Cases – Show appropriate diagrams for at least one boundary condition, such as:
   - `current` at the first song
   - `current` at the last song
   - Deletion of the first or last song

Reference Implementation:

```cpp
#include <iostream>

using namespace std;
// Node class for doubly linked list
class Node {
public:
    int data;
    Node* prev;
    Node* next;

    Node(int val) {
        data = val;
        prev = nullptr;
        next = nullptr;
    }
};

// Doubly linked list class
class DoublyLinkedList {
private:
    Node* head;

public:
    DoublyLinkedList() {
        head = nullptr;
    }

    // Insert node at the end
    void insert(int val) {
        Node* p = new Node(val); // p is used for new node
        if (head == nullptr) {
            head = p;
        } else {
            Node* q = head; // q is used for traversal
            while (q->next != nullptr) {
                q = q->next;
            }
            q->next = p;
            p->prev = q;
        }
    }
```

```
// Delete node by value
void remove(int val) {
    Node *p,*q;
           q = head;

            if (q->data == val) {
            p = head;
            head=head->next;
            head->prev=nullptr;
            delete p;
            return;
            }

            while (q->next != NULL) {
        if (q->next->data == val) {
            p = q->next;
            q->next = p->next;
            p->next->prev=q;
            delete p;
            return;
        }
        q = q->next;
    }

  cout << "Value " << val << " not found in the list.\n";
}

// Display list forward
void displayForward() {
    Node* q = head;
    cout << "Forward: ";
    while (q != nullptr) {
        cout << q->data << " <-> ";
        q = q->next;
    }
    cout << "NULL\n";
}

// Display list backward
void displayBackward() {
    Node* q = head;
    if (q == nullptr) {
        cout << "Backward: NULL\n";
        return;
    }
    // Move to the tail
    while (q->next != nullptr) {
        q = q->next;
    }

    cout << "Backward: ";
```

```
        while (q != nullptr) {
            cout << q->data << " <-> ";
            q = q->prev;
        }
        cout << "NULL\n";
    }
};

// Main function to test the doubly linked list
int main() {
    DoublyLinkedList list;

    list.insert(10);
    list.insert(20);
    list.insert(30);
list.displayForward();//Forward: 10 <-> 20 <-> 30 <-> NULL
list.displayBackward();//Backward:30 <-> 20 <-> 10 <-> NULL

    list.remove(20);
list.displayForward();//Forward: 10 <-> 30 <-> NULL
list.displayBackward();//Backward: 30 <-> 10 <-> NULL

    return 0;
}
```